

AI·웹 애플리케이션 배포 완전 정복

FastAPI · Spring Boot · Vue.js · PostgreSQL · Docker · AWS

허깅페이스 · DockerHub · WinSCP 실무 배포

1장 AI 챗봇 개발 – FastAPI + Gradio HF 배포

이 장에서는 Hugging Face의 고성능 한국어 오픈소스 모델을 활용하여 FastAPI(백엔드)와 Gradio(프론트 엔드 UI)를 결합한 AI 챗봇을 로컬에서 구동하고, 허깅페이스 스페이스(Spaces)에 배포하는 전 과정을 실습합니다. Qwen2.5-1.5B-Instruct 모델을 4-bit 양자화 기술로 경량화하여 일반 PC(VRAM 16GB 이상)에서도 실행할 수 있도록 구성합니다.

1.1 개발 환경 구성

이 절에서는 AI 챗봇 프로젝트를 위한 Python 가상 환경을 생성하고, 웹 서버(FastAPI, Uvicorn), UI(Gradio), AI 구동을 위한 PyTorch 및 Hugging Face 패키지를 설치합니다.

(1) 프로젝트 폴더 및 가상 환경 생성

이 소절에서는 프로젝트 폴더를 만들고 Python 가상 환경을 생성하여 의존성을 격리합니다.

① 폴더 생성 및 가상 환경 활성화

터미널(명령 프롬프트)을 열고 아래 명령어를 순서대로 입력합니다.

터미널(명령 프롬프트)

```
# 프로젝트 폴더 생성 및 이동
mkdir ai-chatbot && cd ai-chatbot

# Python 가상 환경 생성 (Windows)
python -m venv venv
venv\Scripts\activate

# Python 가상 환경 생성 및 활성화 (macOS / Linux)
python3 -m venv venv
source venv/bin/activate
```

mkdir 명령은 폴더를 생성하고, cd 명령은 해당 폴더로 이동합니다. python -m venv venv 명령은 현재 폴더 안에 venv라는 이름의 가상 환경 폴더를 생성합니다. 가상 환경을 활성화하면 프롬프트 앞에 (venv) 표시가 나타나며, 이후 설치되는 패키지는 가상 환경 내부에만 적용됩니다.

(2) 필수 라이브러리 설치

이 소절에서는 웹 서버, UI, AI 구동에 필요한 Python 패키지를 모두 설치합니다.

① 패키지 설치 명령

터미널(명령 프롬프트)

```
# 웹 서버 및 UI 패키지
pip install fastapi uvicorn gradio

# PyTorch (CUDA 12.1 기반 GPU 가속 버전)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# Hugging Face 및 양자화 관련 패키지
pip install transformers accelerate bitsandbytes
```

fastapi는 고성능 Python 웹 프레임워크이며, uvicorn은 FastAPI를 실행하는 ASGI 서버입니다. gradio는 머신러닝 모델의 웹 UI를 빠르게 만들 수 있는 라이브러리입니다. torch(PyTorch)는 딥러닝 연산 라이브러리로 GPU 가속을 지원합니다. transformers는 Hugging Face의 사전 학습된 AI 모델을 불러오고 사용하는 핵심 라이브러리입니다. bitsandbytes는 모델을 4-bit로 양자화하여 메모리 사용량을 크게 줄여주는 라이브러리입니다.

1.2 main.py 소스 코드 작성

이 절에서는 FastAPI 위에 Gradio를 마운트하여 하나의 포트(8000)에서 API 서버와 챗봇 화면이 동시에 구동되는 애플리케이션을 작성합니다.

(1) 전체 소스 코드

이 소절에서는 AI 모델 로딩, 추론 함수, Gradio UI, FastAPI 통합을 모두 포함하는 main.py 파일을 작성합니다.

ai-chatbot/main.py

```
import uvicorn
from fastapi import FastAPI
import gradio as gr
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig

# =====
# 1. FastAPI 애플리케이션 초기화
# =====
app = FastAPI()
```

```

# =====
# 2. AI 모델 설정 (Qwen2.5-7B-Instruct)
# =====
MODEL_ID = "Qwen/Qwen2.5-7B-Instruct"

# 일반 PC(VRAM 8GB 수준)에서 구동하기 위한 4-bit 양자화 설정
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

print("모델을 다운로드하고 불러오는 중입니다... (최초 1회 시간이 걸립니다)")

# 토큰라이저 및 모델 로드
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    quantization_config=bnb_config,
    device_map="auto"
)

# =====
# 3. 챗봇 추론 함수 구현
# =====
def generate_response(message, history):
    messages = []
    messages.append({
        "role": "system",
        "content": "당신은 친절하고 똑똑한 AI 어시스턴트입니다. 한국어로 답변해 주세요."
    })
    for user_msg, bot_msg in history:
        messages.append({"role": "user", "content": user_msg})
        messages.append({"role": "assistant", "content": bot_msg})
    messages.append({"role": "user", "content": message})

    input_ids = tokenizer.apply_chat_template(
        messages,
        add_generation_prompt=True,
        return_tensors="pt"
    ).to(model.device)

    outputs = model.generate(
        input_ids,
        max_new_tokens=1024,
        eos_token_id=model.config.eos_token_id,
        do_sample=True,
        temperature=0.7,
        top_p=0.9,
    )
    response = outputs[0][input_ids.shape[-1]:]
    return tokenizer.decode(response, skip_special_tokens=True)

```

```
# =====
# 4. Gradio 웹 UI 설정
# =====
demo = gr.ChatInterface(
    fn=generate_response,
    title="무료 한국어 AI 챗봇 (FastAPI + Gradio)",
    description="Qwen2.5-7B 모델이 백엔드에서 구동되고 있습니다.",
    examples=["안녕? 넌 누구야?", "스마트 모빌리티에 대해 쉽게 설명해줘."],
    theme="soft"
)

# =====
# 5. FastAPI에 Gradio 화면 마운트
# =====
app = gr.mount_gradio_app(app, demo, path="/")

@app.get("/api/health")
def health_check():
    return {"status": "ok", "message": "FastAPI 백엔드가 정상 작동 중입니다."}

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=False)
```

MODEL_ID는 Hugging Face Hub에서 내려받을 모델의 경로입니다. BitsAndBytesConfig는 4-bit 양자화 설정으로, load_in_4bit=True가 핵심 옵션입니다. bnb_4bit_quant_type="nf4"는 NF4(Normal Float 4) 양자화 방식을 사용하며, 4-bit 중 가장 성능이 좋습니다. device_map="auto"는 사용 가능한 GPU에 모델 레이어를 자동으로 분배합니다. generate_response 함수는 Gradio의 ChatInterface에 연결되며, message(현재 입력)와 history(이전 대화 기록)를 받아 AI 응답을 생성합니다. apply_chat_template은 messages 목록을 모델이 이해할 수 있는 형식으로 변환합니다. temperature=0.7은 답변의 다양성을 제어합니다(0에 가까울수록 결정적, 1에 가까울수록 창의적). gr.mount_gradio_app을 통해 Gradio UI가 FastAPI의 루트 경로(/)에 마운트됩니다.

1.3 로컬 서버 실행 및 접속

이 절에서는 작성한 main.py를 uvicorn으로 실행하고 브라우저로 접속하여 챗봇이 동작하는지 확인합니다.

(1) 서버 실행

이 소절에서는 uvicorn 명령으로 FastAPI+Gradio 서버를 시작합니다.

터미널(명령 프롬프트)

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

```
# 실행 결과 (최초 실행 시 모델 다운로드 포함 5~20분 소요)
모델을 다운로드하고 불러오는 중입니다... (최초 1회 시간이 걸립니다)
INFO: Started server process [12345]
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

최초 실행 시 Hugging Face 서버에서 약 14GB 크기의 모델 파일을 다운로드합니다. 네트워크 환경에 따라 5~20분이 소요될 수 있습니다. 이후 실행부터는 로컬 캐시에서 즉시 로드됩니다. 서버가 시작되면 브라우저에서 `http://localhost:8000` 으로 접속하면 Gradio 챗봇 UI가 표시됩니다.

`http://localhost:8000/api/health` 로 접속하면 FastAPI 헬스체크 JSON 응답을 확인할 수 있습니다.

1.4 FastAPI + Gradio 허깅페이스 스페이스 배포

이 절에서는 로컬에서 개발한 FastAPI + Gradio 애플리케이션을 허깅페이스 스페이스(Hugging Face Spaces)에 Docker 방식으로 배포합니다. 허깅페이스는 무료로 AI 애플리케이션을 배포하고 공유할 수 있는 플랫폼입니다.

(1) 프로젝트 파일 구조

이 소절에서는 허깅페이스 배포에 필요한 3개의 파일 구조를 설명합니다.

```
ai-chatbot/
├── main.py          # FastAPI + Gradio 메인 코드
├── requirements.txt  # 의존성 패키지 목록
└── Dockerfile       # 허깅페이스 배포용 도커 설정
```

(2) requirements.txt 작성

이 소절에서는 배포에 필요한 Python 패키지 목록을 작성합니다.

ai-chatbot/requirements.txt

```
fastapi
uvicorn
gradio
transformers
accelerate
bitsandbytes
torch
```

허깅페이스 스페이스 배포용 requirements.txt는 최소한의 패키지만 명시합니다. 로컬 개발에서 사용한 torch, transformers 등의 AI 패키지는 모델 서빙 목적이 아닌 UI 데모용이므로 여기서는 제외합니다. AI 모델이 필요한 경우 별도로 추가합니다.

(3) Dockerfile 작성

이 소절에서는 허깅페이스 스페이스에서 FastAPI를 안정적으로 구동하기 위한 Dockerfile을 작성합니다. 허깅페이스는 기본적으로 7860 포트를 외부로 노출합니다.

ai-chatbot/Dockerfile

```
# Python 3.12 슬림 이미지 사용
FROM python:3.12-slim

# 작업 디렉토리 설정
WORKDIR /code

# 의존성 파일 복사 및 설치
COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

# 소스 코드 복사
COPY . .

# 허깅페이스 기본 포트(7860) 사용
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

FROM python:3.12-slim은 Python 3.12 경량 이미지를 베이스로 사용합니다. WORKDIR /code는 컨테이너 내부의 작업 디렉터리를 /code로 설정합니다. requirements.txt를 먼저 복사하고 설치하는 이유는 도커 레이어 캐시를 활용하여 소스 코드만 변경될 때는 pip install을 다시 실행하지 않도록 하기 위함입니다. 허깅페이스 스페이스는 7860 포트를 외부에 노출하므로 uvicorn 실행 시 --port 7860을 지정합니다.

(4) 허깅페이스 스페이스 생성 및 파일 업로드

이 소절에서는 허깅페이스 계정에서 새 스페이스를 생성하고 파일을 업로드하여 배포를 완료합니다.

① 스페이스 생성

허깅페이스(<https://huggingface.co>)에 로그인한 후 우측 상단 프로필 → [New Space]를 클릭합니다. Space 이름을 입력합니다(예: fastapi-gradio-chatbot). Space SDK 선택 화면에서 반드시 Gradio가 아닌 Docker를 선택합니다. Blank 템플릿을 선택하고, 하드웨어는 무료인 CPU basic을 선택한 뒤 [Create

Space]를 클릭합니다.

② 파일 업로드

생성된 Space 페이지의 [Files] 탭으로 이동합니다. [Add file] → [Upload files]를 클릭하여 로컬에서 작성한 main.py, requirements.txt, Dockerfile 세 파일을 선택하고 업로드합니다. 하단의 'Commit changes to main'을 클릭하여 파일을 저장합니다.

③ 빌드 확인

파일을 업로드하면 허깅페이스가 자동으로 Docker 이미지를 빌드하기 시작합니다(상태: Building). 1~2분 후 상태가 Running으로 바뀌면 앱 탭에서 Gradio UI를 확인할 수 있습니다. 앱 URL 뒤에 /docs를 붙이면 (예: <https://유저명-스페이스명.hf.space/docs>) FastAPI의 Swagger UI를 확인할 수 있습니다.

2장 AWS EC2 기본 배포 설정

이 장에서는 Amazon Web Services(AWS) EC2 인스턴스를 생성하고, 고정 IP(탄력적 IP)를 할당하며, SSH를 통해 원격 접속하고, Docker 환경을 구축하는 기본 설정을 실습합니다. 이 장의 내용은 이후 모든 AWS 배포의 공통 기반이 됩니다.

2.1 EC2 인스턴스 생성

이 절에서는 AWS 콘솔에서 Ubuntu 기반 EC2 인스턴스를 생성합니다.

(1) 인스턴스 시작 설정

이 소절에서는 EC2 인스턴스 생성에 필요한 각 항목을 설정합니다.

① AWS 콘솔 접속 및 EC2 이동

AWS Management Console(<https://console.aws.amazon.com>)에 로그인합니다. 우측 상단 리전이 '서울 (ap-northeast-2)'인지 확인합니다. 상단 검색창에 EC2를 입력하고 클릭하여 EC2 대시보드로 이동합니다. [인스턴스 시작(Launch instance)] 주황색 버튼을 클릭합니다.

설정 항목	권장 값
이름	my-fullstack-app (프로젝트 식별명)
OS(AMI)	Ubuntu Server 22.04 LTS 또는 24.04 LTS (프리 티어 사용 가능)
인스턴스 유형	t2.micro (프리 티어 무료) 또는 t3.small (AI 서비스)
키 페어	[새 키 페어 생성] → RSA → .pem 형식 → 안전한 곳에 보관 필수
보안 그룹	SSH(22), HTTP(80), HTTPS(443) 허용 체크

2.2 고정 IP(탄력적 IP) 할당

이 절에서는 EC2 인스턴스에 고정 IP를 부여합니다. 기본 EC2는 재시작할 때마다 IP가 바뀌므로, 웹 서비스 운영을 위해 탄력적 IP(Elastic IP)를 할당해야 합니다.

(1) 탄력적 IP 발급 및 연결

이 소절에서는 탄력적 IP를 발급하고 인스턴스에 연결합니다.

① 할당 및 연결 절차

EC2 대시보드 왼쪽 메뉴에서 [네트워크 및 보안] → [탄력적 IP(Elastic IPs)]를 클릭합니다. [탄력적 IP 주소 할당] 버튼 클릭 후 하단 [할당]을 누릅니다. 생성된 IP 주소를 선택하고 [작업] → [탄력적 IP 주소 연결]을 클릭합니다. 인스턴스 입력란에서 생성한 EC2 인스턴스를 선택하고 프라이빗 IP를 선택한 뒤 [연결]을 누릅니다. 화면에 표시된 탄력적 IP 주소를 별도로 메모합니다.

2.3 SSH를 통한 EC2 서버 접속

이 절에서는 다운로드한 키 페어(.pem)를 사용하여 EC2 서버에 원격으로 접속합니다.

(1) macOS / Linux에서 접속

이 소절에서는 macOS와 Linux 터미널에서 SSH로 EC2에 접속합니다.

터미널(명령 프롬프트)

```
# .pem 파일 권한 설정 (최초 1회, 이 설정 없으면 접속 거부됨)
chmod 400 my-app-key.pem
```

```
# EC2 서버 SSH 접속 (탄력적 IP 주소를 실제 값으로 변경)
ssh -i "my-app-key.pem" ubuntu@[탄력적_IP_주소]
```

```
# 최초 접속 시 표시되는 확인 메시지
Are you sure you want to continue connecting (yes/no)? yes
```

```
# 접속 성공 후 표시
ubuntu@ip-172-xx-xx-xx:~$
```

chmod 400은 키 파일을 소유자만 읽을 수 있도록 권한을 제한합니다. 이 설정이 없으면 SSH가 보안 이유로 접속을 거부합니다. ubuntu@[IP]에서 ubuntu는 Ubuntu AMI의 기본 관리자 계정 이름입니다. 접속 성공 시 ubuntu@ip-xxx... 형태의 프롬프트가 표시됩니다.

(2) Windows에서 접속

이 소절에서는 Windows PowerShell 또는 CMD에서 SSH로 EC2에 접속합니다.

PowerShell / CMD

```
# Windows 10 이상은 별도 도구 없이 SSH 사용 가능
ssh -i "my-app-key.pem" ubuntu@[탄력적_IP_주소]
```

2.4 Docker 엔진 설치

이 절에서는 EC2 서버에 Docker Engine과 Docker Compose 플러그인을 설치합니다. Docker를 설치하면 별도로 Java, Python, Nginx 등을 서버에 직접 설치할 필요 없이 컨테이너로 모든 환경을 구성할 수 있습니다.

(1) Docker 설치 명령어

이 소절에서는 Ubuntu에 Docker 공식 저장소를 등록하고 Docker Engine을 설치합니다. EC2에 SSH 접속한 상태에서 아래 명령어를 순서대로 실행합니다.

EC2 서버 내부

```
# 1. 시스템 패키지 업데이트
sudo apt update && sudo apt upgrade -y

# 2. Docker 필수 패키지 설치
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common

# 3. Docker 공식 GPG 키 추가
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# 4. Docker 저장소 등록
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 5. Docker 엔진 및 Compose 플러그인 설치
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# 6. ubuntu 유저를 docker 그룹에 추가 (sudo 없이 docker 명령 사용)
sudo usermod -aG docker $USER
```

각 단계를 설명합니다. 1단계는 시스템 패키지 목록을 최신 상태로 갱신합니다. 2단계는 Docker 설치에 필요한 보안·네트워크 도구를 설치합니다. 3단계는 Docker 공식 GPG 서명 키를 추가합니다. 4단계는 Docker 공식 저장소 주소를 시스템에 등록합니다. 5단계는 Docker Engine, CLI, containerd, Docker

Compose 플러그인을 설치합니다. 6단계는 현재 로그인 중인 ubuntu 계정을 docker 그룹에 추가합니다. 이 설정을 적용하려면 서버 접속을 종료하고 다시 접속해야 합니다.

① Docker 설치 확인

접속을 종료(exit)하고 다시 SSH 접속한 후 아래 명령으로 Docker가 정상 설치되었는지 확인합니다.

EC2 서버 내부

```
# 재접속 후 Docker 설치 확인
docker --version
docker compose version

# 실행 결과
Docker version 27.x.x, build xxxxxxxx
Docker Compose version v2.x.x

# 기본 컨테이너 실행 테스트
docker ps

# 결과 (빈 테이블 = 정상)
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

2.5 docker-compose.yml 작성 및 서비스 실행

이 절에서는 DockerHub에 올려둔 이미지를 사용하여 전체 서비스를 한 번에 실행하는 docker-compose.yml을 작성하고 실행합니다.

(1) docker-compose.yml 파일 생성

이 소절에서는 EC2 서버에서 직접 docker-compose.yml 파일을 작성합니다.

EC2 서버 내부

```
nano docker-compose.yml
```

```
~/docker-compose.yml (EC2 서버)
```

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
```

```

    POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  backend:
    image: your_docker_id/my-backend:v1
    environment:
      DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb
    depends_on:
      - db
    restart: always

  frontend:
    image: your_docker_id/my-frontend:v1
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always

volumes:
  pgdata:

```

```

# 저장: Ctrl + O → Enter
# 종료: Ctrl + X

```

your_docker_id 부분을 본인의 실제 Docker Hub 계정 ID로 변경합니다. services 아래 각 항목이 하나의 컨테이너입니다. db 서비스는 PostgreSQL 공식 이미지를 사용하며, volumes를 통해 데이터를 컨테이너 재 시작 후에도 유지합니다. backend는 DockerHub에서 이미지를 가져오며, depends_on: db를 통해 DB가 먼저 시작된 후 백엔드가 기동됩니다. frontend는 80번 포트를 외부에 노출합니다.

(2) 서비스 실행 및 확인

이 소절에서는 docker compose 명령으로 전체 서비스를 백그라운드에서 실행하고 상태를 확인합니다.

EC2 서버 내부

```

# 백그라운드(-d)에서 컨테이너 실행 (이미지가 없으면 DockerHub에서 자동 다운로드)
docker compose up -d

# 실행 상태 확인
docker compose ps

# 결과 (모두 Up 또는 running 상태여야 정상)
NAME          IMAGE          STATUS
db            postgres:15    Up 30 seconds

```

backend	your_id/my-backend:v1	Up 25 seconds
frontend	your_id/my-frontend:v1	Up 20 seconds

docker compose up -d 명령은 docker-compose.yml에 정의된 모든 서비스를 백그라운드(-d, detached mode)에서 실행합니다. 이미지가 로컬에 없으면 DockerHub에서 자동으로 내려받습니다. docker compose ps로 모든 컨테이너 상태가 'Up' 또는 'running'인지 확인합니다. 이후 브라우저에서 [http://\[탄력적_IP_주소\]](http://[탄력적_IP_주소]) 로 접속하면 배포된 웹 서비스를 확인할 수 있습니다.

3장 WinSCP를 활용한 DockerHub+AWS 배포

이 장에서는 Windows 환경에서 GUI 기반 SFTP 클라이언트인 WinSCP를 사용하여 AWS EC2 서버에 접속하고, DockerHub 이미지를 활용하여 전체 풀스택 서비스를 배포합니다. 명령줄 인터페이스(CLI)에 익숙하지 않은 분도 마우스 드래그 앤 드롭으로 파일을 전송하고 WinSCP 내장 터미널로 명령어를 실행할 수 있습니다.

3.1 WinSCP 설치 및 EC2 접속

이 절에서는 WinSCP를 설치하고 AWS EC2 서버에 SFTP로 접속합니다.

(1) WinSCP 설치

이 소절에서는 WinSCP를 다운로드하여 설치합니다. WinSCP는 <https://winscp.net> 에서 무료로 다운로드할 수 있습니다.

① 접속 설정

WinSCP를 설치하고 실행하면 로그인 창이 나타납니다. 다음 표를 참고하여 접속 정보를 입력합니다.

항목	값
파일 프로토콜	SFTP
호스트 이름	AWS 탄력적 IP 주소 (예: 54.180.xx.xx)
포트 번호	22
사용자 이름	ubuntu
비밀번호	비워둠 (키 파일 사용)

(2) .pem 키 파일 등록

이 소절에서는 EC2 접속에 필요한 .pem 키 파일을 WinSCP에 등록합니다.

① 키 파일 등록 방법

로그인 창 하단의 [고급(Advanced...)] 버튼을 클릭합니다. 왼쪽 메뉴에서 [SSH] → [인증(Authentication)]

을 클릭합니다. '개인 키 파일(Private key file)' 우측의 [...] 버튼을 누릅니다. 파일 선택 창에서 파일 형식을 '모든 파일(*.*)'로 변경한 뒤 다운로드한 .pem 파일을 선택합니다. 'PuTTY 형식(.ppk)으로 변환하시겠습니까?' 안내창이 뜨면 [확인]을 누르고 저장합니다. [확인]으로 고급 창을 닫고, [저장]으로 세션을 저장한 뒤 [로그인]을 클릭합니다.

접속에 성공하면 왼쪽 창에는 내 PC의 파일 시스템이, 오른쪽 창에는 AWSEC2 서버의 파일 시스템이 표시 됩니다. 이제 파일을 마우스로 드래그 앤 드롭하여 서버로 전송할 수 있습니다.

3.2 WinSCP 터미널로 Docker 설치

이 절에서는 WinSCP 내장 터미널(또는 PuTTY)을 사용하여 EC2 서버에 Docker를 설치합니다.

(1) 터미널 열기

이 소절에서는 WinSCP에서 터미널(명령창)을 여는 방법을 설명합니다.

WinSCP 상단 톨바에서 [터미널 열기] 아이콘(검은색 화면 모양)을 클릭하거나, 단축키 Ctrl+T를 누릅니다. 또는 [PuTTY에서 열기](단축키 Ctrl+P)를 눌러 PuTTY 터미널을 사용할 수도 있습니다. 터미널 창이 열리면 EC2 서버에서 명령어를 직접 입력할 수 있습니다.

(2) Docker 설치 명령 실행

이 소절에서는 WinSCP 터미널 창에서 Docker를 설치합니다. 아래 명령어를 복사하여 터미널 창에 붙여넣고 엔터를 누릅니다.

WinSCP 터미널 (EC2 서버)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
```


설치 완료 후 반드시 WinSCP 프로그램을 완전히 종료(X 버튼)하고 다시 열어서 재접속합니다. docker 그룹 권한이 새 세션에 적용됩니다. 재접속 후 터미널에서 `docker ps` 명령을 실행하여 오류 없이 빈 테이블이 출력되면 Docker 설치가 완료된 것입니다.

3.3 docker-compose.yml 파일 전송 및 배포

이 절에서는 로컬 PC에서 작성한 `docker-compose.yml` 파일을 WinSCP로 EC2 서버에 전송하고 서비스를 실행합니다.

(1) docker-compose.yml 작성 (로컬 PC)

이 소절에서는 배포에 사용할 `docker-compose.yml` 파일을 로컬 PC에서 작성합니다.

로컬 PC: docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  backend:
    image: your_docker_id/my-backend:v1
    environment:
      DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb
    depends_on:
      - db
    restart: always

  frontend:
    image: your_docker_id/my-frontend:v1
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always

volumes:
  pgdata:
```

(2) WinSCP로 파일 전송

이 소절에서는 WinSCP GUI를 사용하여 docker-compose.yml 파일을 EC2 서버로 전송합니다.

① 파일 드래그 앤 드롭

WinSCP 왼쪽 창(내 PC)에서 작성한 docker-compose.yml 파일이 있는 폴더로 이동합니다. 오른쪽 창(AWS EC2 서버)의 경로가 /home/ubuntu인지 확인합니다. 왼쪽 창의 docker-compose.yml 파일을 마우스로 클릭하여 오른쪽 창으로 드래그 앤 드롭합니다. 업로드 확인 창이 뜨면 [확인]을 클릭합니다.

(3) 서비스 실행

이 소절에서는 WinSCP 터미널에서 docker compose 명령으로 서비스를 실행합니다.

WinSCP 터미널 (EC2 서버)

```
# 파일 전송 확인  
ls -la
```

```
# 서비스 백그라운드 실행  
docker compose up -d
```

```
# 실행 상태 확인  
docker compose ps
```

```
# 결과 (모두 Up 상태여야 정상)  
NAME          IMAGE                STATUS  
db            postgres:15         Up 30 seconds  
backend       your_id/my-backend:v1 Up 25 seconds  
frontend     your_id/my-frontend:v1 Up 20 seconds
```

docker compose up -d 실행 시 DockerHub에서 이미지를 자동으로 내려받습니다. 모든 컨테이너의 Status가 'Up'이면 배포가 완료된 것입니다. 브라우저에서 [http://\[탄력적_IP_주소\]](http://[탄력적_IP_주소]) 로 접속하여 서비스를 확인합니다.

4장 HTML5 정적 웹 애플리케이션 AWS 배포

이 장에서는 순수 HTML5, CSS3, JavaScript로 구성된 정적 웹 애플리케이션을 AWS EC2에 배포합니다. 별도의 백엔드 서버나 Docker 없이 Nginx 웹 서버만 설치하여 파일을 서빙하는 가장 간단한 배포 방식입니다.

4.1 EC2 서버에 Nginx 설치

이 절에서는 EC2 서버에 접속하여 Nginx 웹 서버를 설치하고, HTML 파일을 업로드할 수 있도록 폴더 권한을 설정합니다.

(1) Nginx 설치 및 폴더 권한 설정

이 소절에서는 apt 패키지 관리자를 사용하여 Nginx를 설치하고 /var/www/html 폴더의 소유권을 변경합니다.

EC2 서버 내부 (SSH 또는 WinSCP 터미널)

```
# 1. 시스템 패키지 업데이트
sudo apt update

# 2. Nginx 웹 서버 설치
sudo apt install nginx -y

# 3. /var/www/html 폴더 소유권을 ubuntu 유저에게 부여
# (WinSCP로 파일을 드래그 앤 드롭으로 넣기 위해 필요)
sudo chown -R ubuntu:ubuntu /var/www/html
```

sudo apt install nginx -y는 Nginx 패키지를 자동 확인(-y) 옵션과 함께 설치합니다. /var/www/html은 Nginx가 기본적으로 정적 파일을 제공하는 디렉터리입니다. sudo chown -R ubuntu:ubuntu /var/www/html 명령은 이 폴더의 소유권을 ubuntu 계정으로 변경합니다. 이 설정이 없으면 WinSCP로 파일을 업로드할 때 권한 오류가 발생합니다.

4.2 HTML 파일 업로드 (WinSCP)

이 절에서는 WinSCP GUI를 사용하여 로컬 PC의 HTML/CSS/JS 파일을 EC2 서버의 Nginx 폴더로 전송합니다.

(1) 기본 파일 삭제 및 웹 파일 업로드

이 소절에서는 WinSCP 오른쪽 창에서 서버의 `/var/www/html` 폴더로 이동하고, 파일을 업로드합니다.

① 서버 폴더 이동

WinSCP 오른쪽 창(AWS 서버) 상단의 경로를 클릭하거나 [...] 폴더 아이콘을 클릭하여 최상위 /(루트) 경로로 이동합니다. `var` → `www` → `html` 순서로 폴더를 더블클릭하여 `/var/www/html` 경로로 이동합니다.

② 기본 파일 삭제

`/var/www/html` 폴더 안에 있는 `index.nginx-debian.html` 파일을 우클릭하여 [삭제]합니다. 이 파일은 Nginx 설치 시 자동으로 생성되는 기본 환영 페이지 파일입니다.

③ HTML 파일 업로드

WinSCP 왼쪽 창(내 PC)에서 배포할 웹 애플리케이션 폴더로 이동합니다. `index.html`, `css` 폴더, `js` 폴더, `images` 폴더 등 웹 서비스에 필요한 모든 파일을 선택하여 오른쪽 창(`/var/www/html`)으로 드래그 앤 드롭합니다. 업로드 확인 창이 뜨면 [확인]을 클릭합니다.

파일 업로드가 완료되면 브라우저에서 `http://[탄력적_IP_주소]` 로 접속합니다. 업로드한 `index.html` 파일이 시작 페이지로 표시되며, CSS와 JavaScript가 정상적으로 적용된 웹 애플리케이션이 전 세계 어디서든 접속 가능한 상태로 배포됩니다.

5장 Vue.js 빌드 및 AWS 배포

이 장에서는 Vue.js 싱글 페이지 애플리케이션(SPA)을 로컬에서 빌드하고 AWS EC2의 Nginx를 통해 배포합니다. SPA 특성상 새로고침 시 404 에러가 발생하지 않도록 Nginx 설정도 함께 진행합니다.

5.1 Vue.js 프로젝트 빌드

이 절에서는 Vue.js 프로젝트를 배포용 정적 파일로 빌드합니다. 빌드 결과물은 dist 폴더에 생성됩니다.

(1) npm run build 실행

이 소절에서는 로컬 PC에서 Vue.js 프로젝트를 빌드합니다.

로컬 PC 터미널

```
# Vue.js 프로젝트 폴더로 이동
cd my-vue-project
```

```
# 프로덕션 빌드 실행
npm run build
```

```
# 빌드 완료 후 dist 폴더 구조
dist/
├── index.html
├── assets/
│   ├── index-xxxxxx.js
│   └── index-xxxxxx.css
```

npm run build 명령은 Vite 또는 Webpack을 사용하여 개발용 코드를 브라우저가 읽기 좋게 최적화·압축합니다. 빌드가 완료되면 프로젝트 폴더 안에 dist 폴더가 생성됩니다. dist 폴더 내부의 모든 파일이 서버에 배포될 파일입니다.

5.2 EC2 서버에 Nginx 설치 및 파일 업로드

이 절에서는 EC2에 Nginx를 설치하고 Vue.js 빌드 파일을 업로드합니다.

(1) Nginx 설치 및 폴더 권한

EC2 서버 내부

```
sudo apt update
sudo apt install nginx -y
sudo chown -R ubuntu:ubuntu /var/www/html
```

(2) 빌드 파일 업로드 (WinSCP)

이 소절에서는 빌드된 dist 폴더 내부의 파일을 서버로 전송합니다.

WinSCP 오른쪽 창에서 /var/www/html로 이동하여 기존 index.nginx-debian.html 파일을 삭제합니다. 왼쪽 창에서 Vue.js 프로젝트의 dist 폴더 안으로 들어갑니다. 주의: dist 폴더 자체가 아닌 dist 폴더 내부의 index.html과 assets 폴더를 선택해야 합니다. 선택한 파일을 오른쪽 창(/var/www/html)으로 드래그 앤 드롭합니다.

5.3 Nginx Vue Router 설정 (404 에러 방지)

이 절에서는 Vue Router의 history 모드를 지원하도록 Nginx 설정을 수정합니다. 이 설정이 없으면 Vue Router 경로에서 새로고침 시 404 에러가 발생합니다.

(1) Nginx 설정 파일 수정

이 소절에서는 nano 편집기로 Nginx 기본 설정 파일을 수정합니다.

EC2 서버 내부

```
sudo nano /etc/nginx/sites-available/default
```

편집기가 열리면 location / { ... } 블록을 찾아 아래와 같이 수정합니다.

/etc/nginx/sites-available/default (수정 전)

```
location / {
    try_files $uri/ =404;
}
```

/etc/nginx/sites-available/default (수정 후)

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

```
# 저장: Ctrl + O → Enter
```

```
# 종료: Ctrl + X
```

```
# Nginx 재시작하여 설정 적용
```

```
sudo systemctl restart nginx
```

try_files \$uri \$uri/ /index.html 설정은 요청된 URL에 해당하는 파일이 없을 때 /index.html을 대신 반환합니다. Vue Router가 /about, /products 등의 경로를 처리하는 것은 모두 index.html 내부의 JavaScript에서 이루어지므로, 서버가 항상 index.html을 반환해야 합니다. 설정 후 F5(새로고침)를 눌러도 404 에러 없이 페이지가 정상 유지되면 배포 성공입니다.

6장 Spring Boot 3 빌드 및 AWS 배포

이 장에서는 Spring Boot 3(Java 21 Zulu OpenJDK)로 개발된 백엔드 애플리케이션을 빌드하고, AWS EC2에 직접 설치하여 배포합니다. Spring Boot는 내장 Tomcat을 포함하므로 .jar 파일 하나만 실행하면 됩니다.

6.1 Spring Boot 프로젝트 빌드

이 절에서는 로컬 PC에서 Spring Boot 프로젝트를 배포 가능한 .jar 파일로 빌드합니다.

(1) Gradle 빌드

이 소절에서는 Gradle을 사용하여 Spring Boot 프로젝트를 빌드합니다.

로컬 PC 터미널

```
# Spring Boot 프로젝트 폴더로 이동
cd my-spring-project

# Gradle 빌드 (-x test: 테스트 코드 건너뛰기)
# macOS / Linux
./gradlew clean build -x test

# Windows
gradlew clean build -x test

# 빌드 성공 시 출력
BUILD SUCCESSFUL in 45s
5 actionable tasks: 5 executed

# 빌드 결과물 확인 (plain 없는 파일이 배포용)
build/libs/
├── myproject-0.0.1-SNAPSHOT.jar      ← 배포용
└── myproject-0.0.1-SNAPSHOT-plain.jar ← 제외
```

./gradlew clean build -x test 명령에서 clean은 이전 빌드 결과물을 삭제하고, build는 컴파일과 패키징을 수행합니다. -x test 옵션은 단위 테스트 실행을 건너뛰어 빌드 시간을 줄입니다. 빌드 완료 후 build/libs/ 폴더에 .jar 파일이 생성됩니다. plain이 붙지 않은 파일이 내장 Tomcat을 포함하는 실행 가능한 배포용 파일입니다.

6.2 EC2에 Java 21(Zulu OpenJDK) 설치

이 절에서는 EC2 서버에 Azul Zulu OpenJDK 21을 설치합니다. Spring Boot 3는 Java 17 이상을 필요로 하며, 이 책에서는 Azul Zulu의 Java 21 버전을 사용합니다.

(1) Zulu OpenJDK 21 설치

이 소절에서는 Azul Systems의 공식 저장소를 등록하고 Zulu JDK 21을 설치합니다.

EC2 서버 내부

```
# 1. 필수 패키지 설치
sudo apt update && sudo apt upgrade -y
sudo apt install -y gnupg ca-certificates curl

# 2. Azul 공식 GPG 키 추가
curl -s https://repos.azul.com/azul-repo.key | sudo gpg --dearmor -o /usr/share/keyrings/azul.gpg

# 3. Azul Zulu APT 저장소 등록
echo "deb [signed-by=/usr/share/keyrings/azul.gpg] \
https://repos.azul.com/zulu/apt stable main" | sudo tee /etc/apt/sources.list.d/zulu.list > /dev/null

# 4. Zulu OpenJDK 21 설치
sudo apt update
sudo apt install -y zulu21-jdk

# 5. 설치 확인 (21 버전이 표시되면 성공)
java -version
```

```
# 설치 확인 결과
openjdk version "21.x.x" 2024-xx-xx LTS
OpenJDK Runtime Environment Zulu21.xx+xx-CA (build 21.x.x+xx-LTS)
OpenJDK 64-Bit Server VM Zulu21.xx+xx-CA (build 21.x.x+xx-LTS, mixed mode, sharing)
```

6.3 .jar 파일 업로드 및 실행

이 절에서는 빌드된 .jar 파일을 WinSCP로 EC2에 업로드하고 백그라운드에서 실행합니다.

(1) WinSCP로 .jar 파일 업로드

이 소절에서는 WinSCP를 사용하여 빌드된 .jar 파일을 EC2의 /home/ubuntu 경로로 전송합니다.

WinSCP 왼쪽 창(내 PC)에서 Spring Boot 프로젝트의 build/libs/ 폴더로 이동합니다. myproject-0.0.1-SNAPSHOT.jar 파일을 오른쪽 창(EC2 서버, /home/ubuntu)으로 드래그 앤 드롭합니다.

(2) 백그라운드 무중단 실행

이 소절에서는 nohup 명령으로 .jar 파일을 터미널 종료 후에도 계속 실행되도록 백그라운드에서 시작합니다.

EC2 서버 내부

```
# 1. 파일 업로드 확인
ls -la

# 2. 백그라운드 무중단 실행 (파일명은 본인의 jar 파일명으로 변경)
nohup java -jar myproject-0.0.1-SNAPSHOT.jar &

# 3. 서버 시작 로그 실시간 확인
tail -f nohup.out
```

```
# 시작 성공 로그 예시
. ____ _
 /WW / __' _ _ _(_) _ _ _ W W W W
...
Started MyProjectApplication in 3.456 seconds (process running for 4.123)

# 로그 확인 종료: Ctrl + C
```

nohup(No Hang UP)은 터미널 연결이 끊겨도 프로세스를 계속 실행하도록 합니다. 마지막의 & 기호는 프로세스를 백그라운드에서 실행합니다. 실행 로그는 nohup.out 파일에 저장됩니다. tail -f nohup.out 명령으로 로그를 실시간 확인할 수 있으며, Ctrl+C로 로그 확인을 종료합니다(서버는 계속 실행됨).

6.4 Nginx 리버스 프록시 설정 (80 → 8080)

이 절에서는 Spring Boot의 기본 포트(8080)로 들어오는 요청을 웹 표준 포트(80)에서 받아 전달하는 Nginx 리버스 프록시를 설정합니다.

(1) Nginx 설치 및 설정

EC2 서버 내부

```
sudo apt install nginx -y
sudo nano /etc/nginx/sites-available/default
```

/etc/nginx/sites-available/default (수정 후)

```
location / {
    proxy_pass http://localhost:8080;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}
```

```
# 저장: Ctrl + O → Enter → Ctrl + X
sudo systemctl restart nginx
```

proxy_pass http://localhost:8080은 80번 포트로 들어온 요청을 서버 내부의 8080번 포트(Spring Boot)로 전달합니다. proxy_set_header 설정들은 실제 클라이언트의 IP 주소와 호스트 정보를 백엔드 서버에 전달합니다. 설정 완료 후 브라우저에서 http://[탄력적_IP_주소] (포트 번호 없이)로 접속하면 Spring Boot 애플리케이션이 응답합니다.

7장 Spring Boot + Vue.js + PostgreSQL 배포

이 장에서는 Docker를 사용하지 않고 하나의 EC2 서버에 PostgreSQL(DB), Spring Boot(백엔드), Vue.js+Nginx(프론트엔드)를 직접 설치하여 연결하는 네이티브 배포 방식을 실습합니다. Nginx가 80번 포트에서 요청을 받아 화면 요청은 Vue.js로, API 요청(/api/)은 Spring Boot(8080)로 분기합니다.

7.1 전체 아키텍처

이 절에서는 이 장에서 구성할 시스템의 전체 트래픽 흐름을 설명합니다.

```

브라우저 HTTP 요청 → [Nginx 포트 80]
├── /      → /var/www/html (Vue.js 정적 파일 서빙)
└── /api/  → localhost:8080 (Spring Boot 리버스 프록시)
            └── localhost:5432 (PostgreSQL)
  
```

7.2 로컬 빌드

이 절에서는 배포 전 로컬에서 Spring Boot와 Vue.js를 각각 빌드합니다.

(1) Spring Boot 설정 변경

이 소절에서는 배포용 DB 접속 주소를 localhost로 설정합니다. 서버 한 대에 DB와 백엔드가 함께 실행되므로 localhost를 사용합니다.

src/main/resources/application.yml

```

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: myuser
    password: mypassword
  
```

로컬 PC 터미널

```
./gradlew clean build -x test
```

(2) Vue.js 빌드

이 소절에서는 Vue.js 프론트엔드를 빌드합니다. API 요청이 /api/... 형태인지 미리 확인합니다.

로컬 PC 터미널

```
cd my-vue-project
npm run build
```

7.3 EC2 서버 환경 구축

이 절에서는 EC2에 Nginx, PostgreSQL, Java 21을 한 번에 설치합니다.

(1) 한 번에 설치

EC2 서버 내부

```
sudo apt update && sudo apt upgrade -y

# Nginx, PostgreSQL 설치
sudo apt install nginx postgresql postgresql-contrib -y

# Zulu Java 21 설치
sudo apt install -y gnupg ca-certificates curl
curl -s https://repos.azul.com/azul-repo.key | sudo gpg --dearmor -o /usr/share/keyrings/azul.gpg
echo "deb [signed-by=/usr/share/keyrings/azul.gpg] https://repos.azul.com/zulu/apt stable main" | sudo tee /etc/apt/sources.list.d/zulu.list > /dev/null
sudo apt update && sudo apt install -y zulu21-jdk
```

7.4 PostgreSQL DB 및 사용자 생성

이 절에서는 PostgreSQL에 Spring Boot가 사용할 데이터베이스와 사용자를 생성합니다.

(1) DB 콘솔 접속 및 생성

EC2 서버 내부

```
# postgres 관리자 계정으로 DB 콘솔 진입
sudo -u postgres psql
```

```
-- DB 콘솔 내부에서 실행 (세미콜론 ; 필수)
CREATE DATABASE mydb;
CREATE USER myuser WITH ENCRYPTED PASSWORD 'mypassword';
GRANT ALL PRIVILEGES ON DATABASE mydb TO myuser;
Wq
```

7.5 파일 업로드 및 서비스 실행

이 절에서는 WinSCP로 빌드 파일을 업로드하고 서비스를 실행합니다.

(1) 폴더 권한 설정 및 파일 업로드

EC2 서버 내부

```
# Nginx 폴더 권한 설정
sudo chown -R ubuntu:ubuntu /var/www/html

# Spring Boot 업로드용 폴더 생성
mkdir /home/ubuntu/spring_app
```

WinSCP 오른쪽 창을 /var/www/html 경로로 이동하여 기존 index.nginx-debian.html 파일을 삭제합니다. 왼쪽 창의 Vue.js dist 폴더 내부 파일(index.html, assets/)을 /var/www/html로 드래그 앤 드롭합니다. 다시 오른쪽 창을 /home/ubuntu/spring_app 경로로 이동하고, Spring Boot의 .jar 파일을 드래그 앤 드롭합니다.

(2) Spring Boot 실행

EC2 서버 내부

```
cd /home/ubuntu/spring_app
nohup java -jar myproject-0.0.1-SNAPSHOT.jar &
tail -f nohup.out
```

7.6 Nginx 통합 라우팅 설정

이 절에서는 Nginx가 Vue.js 화면과 Spring Boot API 요청을 올바르게 분기하도록 설정합니다.

(1) Nginx 설정 파일 작성

EC2 서버 내부

```
sudo nano /etc/nginx/sites-available/default
```

/etc/nginx/sites-available/default (전체 교체)

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    # Vue.js 화면 서빙 (새로고침 404 방지 포함)
    location / {
        root /var/www/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }

    # Spring Boot API 포워딩 (/api/ 경로 → 8080 포트)
    location /api/ {
        proxy_pass http://localhost:8080;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
    }
}
```

```
# 저장: Ctrl + O → Enter → Ctrl + X
sudo systemctl restart nginx
```

이 Nginx 설정으로 두 가지 라우팅이 동시에 적용됩니다. 첫째, / 경로는 /var/www/html의 Vue.js 파일을 서빙합니다. try_files \$uri \$uri/ /index.html 덕분에 Vue Router 경로에서 새로고침해도 404 에러가 발생하지 않습니다. 둘째, /api/ 경로는 proxy_pass를 통해 내부의 Spring Boot(8080 포트)로 요청을 전달합니다.

8장 Spring Boot + Vue.js AWS 배포

이 장에서는 Spring Boot, Vue.js, PostgreSQL을 각각 도커 이미지로 만들어 DockerHub에 업로드한 후, AWS EC2에서 docker-compose.yml 파일 하나로 전체 풀스택 서비스를 배포하는 현대적인 프로덕션 배포 방식을 실습합니다.

8.1 Dockerfile 작성 및 DockerHub Push

이 절에서는 Spring Boot와 Vue.js 각각에 대한 Dockerfile을 작성하고 DockerHub에 이미지를 업로드합니다.

(1) Spring Boot Dockerfile

이 소절에서는 Spring Boot 배포용 Dockerfile을 작성합니다.

spring-project/Dockerfile

```
# Java 21 Zulu OpenJDK 공식 이미지 사용
FROM azul/zulu-openjdk:21

WORKDIR /app

# 빌드된 jar 파일을 컨테이너 안으로 복사
COPY build/libs/*-SNAPSHOT.jar app.jar

# Spring Boot 기본 포트
EXPOSE 8080

# 컨테이너 실행 시 jar 파일 구동
CMD ["java", "-jar", "app.jar"]
```

로컬 PC 터미널 (Spring Boot 프로젝트 폴더)

```
# Spring Boot 먼저 빌드
./gradlew clean build -x test

# 도커 이미지 빌드 및 DockerHub Push
docker build -t your_id/spring-backend:v1 .
docker push your_id/spring-backend:v1
```


(2) Vue.js Dockerfile (Nginx 내장)

이 소절에서는 Vue.js를 빌드하고 Nginx로 서빙하는 멀티 스테이지 Dockerfile을 작성합니다.

vue-project/Dockerfile

```
# 1단계: Node.js로 빌드
FROM node:18 AS build-stage
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# 2단계: Nginx로 정적 파일 서빙 + API 리버스 프록시
FROM nginx:alpine

# 빌드 결과물 복사
COPY --from=build-stage /app/dist /usr/share/nginx/html

# Nginx 설정: / → Vue, /api/ → Spring Boot(backend:8080)
RUN echo "server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://backend:8080/api;
    }
}" > /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

로컬 PC 터미널 (Vue.js 프로젝트 폴더)

```
docker build -t your_id/vue-frontend:v1 .
docker push your_id/vue-frontend:v1
```

이 Dockerfile은 멀티 스테이지(Multi-Stage) 빌드 방식을 사용합니다. 1단계(build-stage)에서 Node.js 환경에서 npm run build를 실행하여 dist 폴더를 생성합니다. 2단계에서는 경량 Nginx Alpine 이미지에 1단계의 빌드 결과물만 복사합니다. 최종 이미지에 Node.js 환경이 포함되지 않으므로 이미지 크기가 훨씬 작아집니다. RUN echo의 Nginx 설정에서 proxy_pass http://backend:8080은 Docker Compose 네트워크 내부에서 backend라는 이름의 컨테이너로 요청을 전달합니다.

8.2 EC2 배포용 docker-compose.yml

이 절에서는 EC2 서버에서 사용할 배포용 docker-compose.yml을 작성하고 배포합니다.

(1) docker-compose.yml 작성 및 업로드

이 소절에서는 DockerHub 이미지를 사용하는 배포용 docker-compose.yml을 작성합니다.

docker-compose.yml (로컬 작성 후 WinSCP로 EC2 서버에 전송)

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  backend:
    image: your_id/spring-backend:v1
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mydb
      SPRING_DATASOURCE_USERNAME: myuser
      SPRING_DATASOURCE_PASSWORD: mypassword
    depends_on:
      - db
    restart: always

  frontend:
    image: your_id/vue-frontend:v1
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always

volumes:
  pgdata:
```

EC2 서버 내부

```
docker compose up -d
docker compose ps
```

9장 FastAPI + Vue.js + PostgreSQL AWS 배포

이 장에서는 FastAPI(Python 3.12)를 백엔드로 사용하는 풀스택 애플리케이션을 DockerHub에 올리고 AWS EC2에 배포합니다. Python은 Java와 달리 별도의 컴파일 과정 없이 소스 코드 자체를 도커 이미지로 패키징합니다.

9.1 FastAPI 배포 준비 및 Dockerfile

이 절에서는 FastAPI 프로젝트의 의존성을 추출하고 Dockerfile을 작성합니다.

(1) requirements.txt 생성

이 소절에서는 현재 Python 가상 환경에 설치된 패키지 목록을 requirements.txt 파일로 추출합니다.

로컬 PC 터미널 (FastAPI 프로젝트 폴더, 가상환경 활성화 상태)

```
pip freeze > requirements.txt
```

(2) FastAPI Dockerfile

fastapi-project/Dockerfile

```
# Python 3.12 슬림 버전 이미지
FROM python:3.12-slim

WORKDIR /app

# 패키지 목록 먼저 복사 (레이어 캐시 최적화)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 소스 코드 전체 복사
COPY . .

# FastAPI 기본 포트
EXPOSE 8000

# uvicorn으로 FastAPI 서버 구동
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

로컬 PC 터미널

```
docker build -t your_id/fastapi-backend:v1 .
docker push your_id/fastapi-backend:v1
```

Python FastAPI의 Dockerfile은 소스 코드 원본과 requirements.txt를 함께 이미지에 포함합니다. COPY requirements.txt .을 소스 코드 복사 전에 수행하는 이유는 도커 레이어 캐시를 활용하기 위해서입니다. requirements.txt가 변경되지 않으면 pip install 단계가 캐시에서 재사용되어 빌드 시간이 크게 줄어듭니다.

9.2 Vue.js Dockerfile (FastAPI 연동)

이 절에서는 API 요청을 FastAPI(8000 포트)로 전달하는 Nginx 설정이 포함된 Vue.js Dockerfile을 작성합니다.

(1) Vue.js Dockerfile

vue-project/Dockerfile

```
FROM nginx:alpine

COPY dist /usr/share/nginx/html

# /api/ 요청을 backend 컨테이너(8000 포트)로 전달
RUN echo "server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://backend:8000;
    }
}" > /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

로컬 PC 터미널 (Vue.js 프로젝트 폴더, npm run build 후)

```
docker build -t your_id/vue-frontend:v1 .
docker push your_id/vue-frontend:v1
```

9.3 EC2 배포용 docker-compose.yml

이 절에서는 FastAPI 기반 풀스택 서비스를 EC2에서 실행하는 docker-compose.yml을 작성합니다.

(1) docker-compose.yml 작성

docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  backend:
    image: your_id/fastapi-backend:v1
    environment:
      DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb
    depends_on:
      - db
    restart: always

  frontend:
    image: your_id/vue-frontend:v1
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always

volumes:
  pgdata:
```

EC2 서버 내부

```
docker compose up -d
docker compose ps
```

10장 FastAPI + Vue.js + PostgreSQL 배포

이 장에서는 Docker 없이 하나의 EC2 서버에 PostgreSQL, Python 3.12(FastAPI), Nginx(Vue.js)를 직접 설치하고 연결하는 네이티브 배포 방식을 실습합니다. Python은 소스 코드 원본과 requirements.txt를 서버에 올린 후 가상 환경(venv)을 만들어 실행합니다.

10.1 로컬 준비

이 절에서는 배포 전 로컬 PC에서 FastAPI와 Vue.js를 각각 준비합니다.

(1) FastAPI 준비

이 소절에서는 FastAPI 프로젝트의 DB 접속 주소를 localhost로 설정하고 패키지 목록을 추출합니다.

로컬 PC 터미널 (FastAPI 프로젝트)

```
# 패키지 목록 추출
pip freeze > requirements.txt
```

FastAPI 코드에서 DB 접속 주소를 환경 변수로 읽도록 작성합니다.

fastapi-project/main.py (DB 접속 예시)

```
import os
from sqlalchemy import create_engine

# 환경 변수에서 DB URL 읽기 (없으면 localhost 사용)
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://myuser:mypassword@localhost:5432/mydb"
)
engine = create_engine(DATABASE_URL)
```

(2) Vue.js 빌드

로컬 PC 터미널 (Vue.js 프로젝트)

```
npm run build
```

10.2 EC2 환경 구축

이 절에서는 EC2에 Nginx, PostgreSQL, Python 가상 환경 도구를 설치합니다.

(1) 필수 패키지 설치

EC2 서버 내부

```
sudo apt update && sudo apt upgrade -y
sudo apt install nginx postgresql postgresql-contrib -y
sudo apt install python3-pip python3-venv -y
```

(2) PostgreSQL DB 생성

EC2 서버 내부

```
sudo -u postgres psql
```

```
-- psql 콘솔 내부
CREATE DATABASE mydb;
CREATE USER myuser WITH ENCRYPTED PASSWORD 'mypassword';
GRANT ALL PRIVILEGES ON DATABASE mydb TO myuser;
\q
```

10.3 파일 업로드 및 FastAPI 실행

이 절에서는 WinSCP로 파일을 업로드하고 FastAPI를 가상 환경에서 실행합니다.

(1) 업로드 폴더 준비

EC2 서버 내부

```
sudo chown -R ubuntu:ubuntu /var/www/html
mkdir /home/ubuntu/fastapi_app
```

WinSCP를 사용하여 다음과 같이 파일을 전송합니다. Vue.js: dist 폴더 내부의 모든 파일 → /var/www/html (기존 index.nginx-debian.html 삭제 후). FastAPI: main.py, 소스 코드 폴더들, requirements.txt → /home/ubuntu/fastapi_app (venv, __pycache__ 폴더는 제외).

(2) 가상 환경 생성 및 FastAPI 실행

EC2 서버 내부

```
cd /home/ubuntu/fastapi_app

# 가상 환경 생성
python3 -m venv venv

# 가상 환경 활성화
source venv/bin/activate

# 패키지 설치
pip install -r requirements.txt

# 백그라운드 무중단 실행
nohup uvicorn main:app --host 127.0.0.1 --port 8000 &

# 실행 로그 확인 (Ctrl+C로 종료)
tail -f nohup.out
```

```
# 성공 로그
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000
```

10.4 Nginx 통합 라우팅 설정

이 절에서는 Vue.js 화면과 FastAPI API 요청을 Nginx로 분기합니다.

(1) Nginx 설정

EC2 서버 내부

```
sudo nano /etc/nginx/sites-available/default
```

```
/etc/nginx/sites-available/default
```

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
```



```
# Vue.js 정적 파일 서빙
location / {
    root /var/www/html;
    index index.html index.htm;
    try_files $uri $uri/ /index.html;
}

# FastAPI API 포워딩 (/api/ → 8000 포트)
location /api/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host $http_host;
}
}
```

```
sudo systemctl restart nginx
```

11장 FastAPI + Vue.js + PostgreSQL 도커 배포

이 장에서는 FastAPI, Vue.js, PostgreSQL을 모노레포(Monorepo) 방식으로 구성하고 로컬에서 Docker Compose로 테스트한 뒤 DockerHub를 거쳐 AWS EC2에 배포하는 완전한 파이프라인을 실습합니다.

11.1 프로젝트 구조

이 절에서는 모노레포 방식의 프로젝트 구조를 설명합니다.

```
my_fullstack_app/
├── backend/           # FastAPI (Python)
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/         # Vue.js (Node.js)
│   ├── src/
│   ├── package.json
│   └── Dockerfile
└── docker-compose.yml # 전체 서비스 오케스트레이션
```

11.2 백엔드 예시 코드

이 절에서는 PostgreSQL과 연동하는 FastAPI 예시 코드를 작성합니다.

(1) FastAPI 메인 코드

backend/requirements.txt

```
fastapi
uvicorn
sqlalchemy
psycopg2-binary
```

backend/main.py

```
from fastapi import FastAPI
from sqlalchemy import create_engine, text
import os

app = FastAPI()
```

```
# 환경 변수에서 DB 접속 정보 가져오기 (Docker에서 주입)
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://user:password@localhost:5432/mydb"
)
engine = create_engine(DATABASE_URL)

@app.get("/api/status")
def read_status():
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
            return {
                "status": "ok",
                "db_connection": "success",
                "message": "FastAPI와 PostgreSQL이 연결되었습니다."
            }
    except Exception as e:
        return {"status": "error", "details": str(e)}
```

os.getenv("DATABASE_URL")는 환경 변수에서 DB 접속 문자열을 읽습니다. Docker Compose에서 environment 항목으로 이 값을 컨테이너에 주입합니다. engine.connect()로 DB 연결을 시도하고 SELECT 1 쿼리가 성공하면 정상 연결로 판단합니다.

11.3 프론트엔드 예시 코드

이 절에서는 FastAPI 백엔드를 호출하는 Vue.js 컴포넌트 예시를 작성합니다.

(1) App.vue

frontend/src/App.vue

```
<template>
  <div id="app">
    <h1>풀스택 앱 상태 확인</h1>
    <button @click="checkStatus">서버 및 DB 상태 확인</button>
    <pre v-if="statusMessage">{{ statusMessage }}</pre>
  </div>
</template>

<script>
export default {
  data() {
    return { statusMessage: '' }
  },
  methods: {
```

```

async checkStatus() {
  try {
    // Nginx 리버스 프록시를 통해 /api 요청이 백엔드로 전달됨
    const response = await fetch('/api/status');
    const data = await response.json();
    this.statusMessage = JSON.stringify(data, null, 2);
  } catch (error) {
    this.statusMessage = '서버 통신 실패: ' + error.message;
  }
}
}
}
</script>

```

Vue.js에서 API 호출 시 `http://backend:8000/api/status`처럼 직접 백엔드 주소를 지정하지 않고 `/api/status`처럼 상대 경로를 사용합니다. Nginx가 `/api/` 경로를 감지하여 백엔드 컨테이너로 요청을 전달합니다. 이 방식은 프론트엔드 코드를 수정하지 않고도 백엔드 주소를 변경할 수 있어 유지보수가 편리합니다.

11.4 로컬 테스트 및 DockerHub 배포

이 절에서는 로컬에서 docker compose로 전체 서비스를 테스트하고 DockerHub에 이미지를 Push합니다.

(1) 로컬 테스트용 docker-compose.yml

my_fullstack_app/docker-compose.yml

```

version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb
    depends_on:
      - db

```

```
frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - backend
```

```
volumes:
  pgdata:
```

로컬 PC 터미널

```
# 로컬에서 전체 서비스 테스트
docker compose up

# 테스트 완료 후 DockerHub에 Push
docker login
docker build -t your_id/my-backend:v1 ./backend
docker build -t your_id/my-frontend:v1 ./frontend
docker push your_id/my-backend:v1
docker push your_id/my-frontend:v1
```

12장 Spring Boot + FastAPI + Vue.js 배포

이 장에서는 일반 비즈니스 로직을 담당하는 Spring Boot(메인 백엔드)와 AI 추론 전용 FastAPI(AI 백엔드)를 분리하여 운영하는 마이크로서비스 아키텍처(MSA)를 구성합니다. Nginx가 요청 경로에 따라 Spring Boot 또는 FastAPI로 요청을 분기합니다. 네이티브 방식과 Docker 방식을 모두 실습합니다.

12.1 전체 MSA 아키텍처

이 절에서는 Spring Boot + FastAPI 이중 백엔드 구조의 트래픽 흐름을 설명합니다.

```
브라우저 HTTP 요청 → [Nginx 포트 80]
├── /          → /var/www/html (Vue.js 화면 서빙)
├── /api/       → localhost:8080 (Spring Boot 비즈니스 로직)
└── /ai/        → localhost:8000 (FastAPI AI 추론 로직)
```

Spring Boot (8080) ↔ PostgreSQL (5432)

12.2 네이티브 배포 (Docker 미사용)

이 절에서는 Docker 없이 EC2 서버에 모든 서비스를 직접 설치하고 실행합니다.

(1) 로컬 빌드 준비

이 소절에서는 세 가지 서비스를 각각 빌드 또는 배포 준비합니다.

로컬 PC 터미널

```
# Spring Boot 빌드
cd spring-project
./gradlew clean build -x test

# FastAPI 패키지 목록 추출
cd fastapi-project
pip freeze > requirements.txt

# Vue.js 빌드 (API 경로 확인: /api/ → Spring, /ai/ → FastAPI)
cd vue-project
npm run build
```

(2) EC2 환경 구축

이 소절에서는 EC2에 Nginx, PostgreSQL, Java 21, Python 가상 환경을 설치합니다.

EC2 서버 내부

```
sudo apt update && sudo apt upgrade -y
sudo apt install nginx postgresql postgresql-contrib python3-pip python3-venv -y

# Java 21 설치
sudo apt install -y gnupg ca-certificates curl
curl -s https://repos.azul.com/azul-repo.key | sudo gpg --dearmor -o /usr/share/keyrings/azul.gpg
echo "deb [signed-by=/usr/share/keyrings/azul.gpg] https://repos.azul.com/zulu/apt stable main" | sudo tee /etc/apt/sources.list.d/zulu.list > /dev/null
sudo apt update && sudo apt install -y zulu21-jdk
```

(3) PostgreSQL DB 생성

EC2 서버 내부

```
sudo -u postgres psql
```

```
CREATE DATABASE mydb;
CREATE USER myuser WITH ENCRYPTED PASSWORD 'mypassword';
GRANT ALL PRIVILEGES ON DATABASE mydb TO myuser;
\q
```

(4) 파일 업로드 및 서비스 실행

이 소절에서는 폴더를 생성하고 WinSCP로 파일을 업로드한 후 두 개의 백엔드를 실행합니다.

EC2 서버 내부

```
sudo chown -R ubuntu:ubuntu /var/www/html
mkdir /home/ubuntu/spring_app
mkdir /home/ubuntu/fastapi_app
```

WinSCP로 다음과 같이 파일을 전송합니다. Vue.js dist 내부 파일 → /var/www/html. Spring Boot .jar → /home/ubuntu/spring_app. FastAPI main.py, 소스 폴더, requirements.txt → /home/ubuntu/fastapi_app (venv 제외).

EC2 서버 내부

```
# Spring Boot 실행 (8080 포트)
cd /home/ubuntu/spring_app
nohup java -jar myproject-0.0.1-SNAPSHOT.jar &
tail -f nohup.out
# 시작 확인 후 Ctrl+C

# FastAPI 실행 (8000 포트)
cd /home/ubuntu/fastapi_app
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
nohup uvicorn main:app --host 127.0.0.1 --port 8000 &
tail -f nohup.out
```

(5) Nginx MSA 라우팅 설정

이 소절에서는 Nginx가 경로에 따라 Spring Boot와 FastAPI로 요청을 분기하도록 설정합니다.

EC2 서버 내부

```
sudo nano /etc/nginx/sites-available/default
```

```
/etc/nginx/sites-available/default
```

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    # Vue.js 화면 (새로고침 404 방지 포함)
    location / {
        root /var/www/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }

    # Spring Boot 비즈니스 API (8080 포트)
    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
    }

    # FastAPI AI 추론 API (8000 포트)
    location /ai/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
    }
}
```



```
}
}
```

```
sudo systemctl restart nginx
```

12.3 Docker 기반 MSA 배포 (DockerHub 활용)

이 절에서는 Docker를 사용하여 네 개의 컨테이너(PostgreSQL, Spring Boot, FastAPI, Vue.js+Nginx)로 구성된 MSA 시스템을 배포합니다.

(1) Spring Boot Dockerfile 및 Push

spring-project/Dockerfile

```
FROM azul/zulu-openjdk:21
WORKDIR /app
COPY build/libs/*-SNAPSHOT.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

로컬 PC 터미널

```
./gradlew clean build -x test
docker build -t your_id/spring-backend:v1 .
docker push your_id/spring-backend:v1
```

(2) FastAPI Dockerfile 및 Push

fastapi-project/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

로컬 PC 터미널

```
docker build -t your_id/fastapi-backend:v1 .
docker push your_id/fastapi-backend:v1
```

(3) Vue.js Dockerfile (이중 프록시)

이 소절에서는 /api/는 Spring Boot로, /ai/는 FastAPI로 전달하는 Nginx 설정이 포함된 Vue.js Dockerfile 을 작성합니다.

vue-project/Dockerfile

```
FROM nginx:alpine
COPY dist /usr/share/nginx/html

RUN echo "server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://spring-backend:8080;
    }
    location /ai/ {
        proxy_pass http://fastapi-backend:8000;
    }
}" > /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

로컬 PC 터미널

```
npm run build
docker build -t your_id/vue-frontend:v1 .
docker push your_id/vue-frontend:v1
```

(4) EC2 배포용 docker-compose.yml

이 소절에서는 4개 컨테이너를 한 번에 실행하는 docker-compose.yml을 작성합니다. WinSCP로 EC2의 /home/ubuntu에 업로드합니다.

docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
```

```

volumes:
  - pgdata:/var/lib/postgresql/data
restart: always

spring-backend:
  image: your_id/spring-backend:v1
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mydb
    SPRING_DATASOURCE_USERNAME: myuser
    SPRING_DATASOURCE_PASSWORD: mypassword
  depends_on:
    - db
  restart: always

fastapi-backend:
  image: your_id/fastapi-backend:v1
  environment:
    DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb
  depends_on:
    - db
  restart: always

frontend:
  image: your_id/vue-frontend:v1
  ports:
    - "80:80"
  depends_on:
    - spring-backend
    - fastapi-backend
  restart: always

volumes:
  pgdata:

```

EC2 서버 내부

```

docker compose up -d
docker compose ps

```

```

# 성공 시 4개 컨테이너 모두 Up 상태
NAME                IMAGE                                STATUS
db                  postgres:15                         Up 30 seconds
spring-backend      your_id/spring-backend:v1          Up 25 seconds
fastapi-backend     your_id/fastapi-backend:v1         Up 23 seconds
frontend            your_id/vue-frontend:v1            Up 20 seconds

```

4개 컨테이너가 모두 Up 상태이면 M SA 기반 풀스택 배포가 성공적으로 완료된 것입니다. 브라우저에서 `http://[탄력적_IP_주소]`로 접속하면 Vue.js 화면이 표시되고, 일반 로직은 Spring Boot가, AI 추론 로직은 FastAPI가 처리하며, 각각 PostgreSQL과 통신하는 완전한 4중 컨테이너 M SA 시스템이 작동합니다.